
TEMPO: Regime-Aware Operator Learning with Locally Adapted POD Bases

Anonymous Author(s)

Affiliation

Address

email

Abstract

Operator learning methods typically approximate the solution map of a physical system with a single global model—an assumption that fails when solutions change character fundamentally across the parameter space.

We propose TEMPO: a framework that identifies latent dynamical regimes in snapshot data via Expectation-Maximization, builds a dedicated Neural-POD basis for each, and learns to route new inputs to the right regime at inference time. On multi-regime benchmarks, TEMPO achieves substantially lower errors than POD-DeepONet and vanilla DeepONet, with interpretable structure that reflects the underlying physics.

Keywords: Operator learning, DeepONet, Proper Orthogonal Decomposition, Expectation-Maximization, Gaussian mixture models, Regime discovery

1 Introduction

Operator learning addresses the approximation of mappings between infinite-dimensional function spaces, arising naturally in the solution of parametric partial differential equations (PDEs) governing physical systems [1]. This paradigm has demonstrated substantial promise across both theoretical analysis [1, 2] and practical applications including weather modeling, quantum systems, fluid and sound dynamics [3–6].

Among the most prominent architectures, Deep Operator Networks (DeepONet) [2] approximate operators via paired branch and trunk networks, where the branch encodes the input function and the trunk encodes the query location. The Fourier Neural Operator (FNO) [7] independently parameterizes the solution operator in the spectral domain, offering resolution-invariance and strong empirical performance. POD-DeepONet [8] improves upon standard DeepONet by replacing the learned trunk network with a fixed basis obtained from Proper Orthogonal Decomposition (POD) of training snapshots, yielding more compact and interpretable representations.

However, a single global POD basis is inherently limited for problems whose solutions exhibit qualitatively distinct dynamical regimes—such as solutions transitioning between laminar and turbulent behavior, or dispersive waves with parameter-dependent structure. In such settings, the optimal basis varies across the solution manifold, and a single linear subspace cannot represent the full solution diversity.

Several approaches enrich the POD basis to overcome this limitation. Sharma and Shankar [9] update the DeepONet trunk with multiple complementary bases—precomputed POD modes and a partition-of-unity mixture-of-experts (PoU-MoE) trunk, trained jointly with a single branch. On 2D and 3D problems with sharp spatial gradients, the POD-PoU ensemble achieves up to $2\times$ lower errors over POD-DeepONet [8] and $2\text{--}4\times$ over vanilla DeepONet.

Moreover, the linear structure of the POD basis is insufficient for strongly nonlinear solution manifolds. Mou et al. [10] propose Neural-POD, which constructs nonlinear orthogonal basis functions via iterative residual minimization analogous to Gram–Schmidt orthogonalization, representing each spatial mode and temporal component as neural networks trained sequentially on the residual variance.

Yet both approaches enrich the basis without identifying the underlying dynamical regimes present in the data. This idea has been pursued in the reduced order modeling (ROM) literature, where snapshot ensembles are partitioned into clusters and a separate local POD basis is constructed for each [11, 12]. More recently, Daniel et al. [13] extended this paradigm by clustering solution subspaces on the Grassmann manifold and training a deep neural network to select the appropriate local model from a dictionary at inference time. However, these methods operate offline on fixed snapshot sets, employ hard cluster assignments, and do not integrate regime discovery with operator learning in an end-to-end trainable architecture.

In this work, we propose **TEMPO** (*Regime-Aware Operator Learning with Locally Adapted POD Bases*). Our method integrates regime discovery directly into operator learning: a Gaussian mixture model over Neural-POD coefficient space identifies latent dynamical regimes via soft EM, constructing a dedicated locally adapted basis for each. At inference time, a learned gating network selects among regimes, producing predictions that are accurate across the full parameter space.

2 Problem Statement

The behavior of many physical systems is governed by a set of parameters $\kappa \in \mathcal{K} \subset \mathbb{R}^{d_\kappa}$: viscosity controls whether flow is laminar or turbulent, boundary forcing shapes wave patterns, material coefficients determine whether stress propagates smoothly or develops shocks. As κ varies across the parameter space $\mathcal{K}$, the corresponding solutions can change not just quantitatively but qualitatively, demanding fundamentally different representations in each regime. Predicting these solutions accurately and instantly, without re-running the underlying simulator, is the central problem we address.

Let $\Omega \subset \mathbb{R}^d$ be a bounded spatial domain and $\mathcal{T} \subset \mathbb{R}_{\geq 0}$ a time interval.

We are given a dataset of N solution trajectories:

$$\mathcal{D} = \{ (u_0^{(i)}, \kappa^{(i)}, s^{(i)}) \}_{i=1}^N, \quad s^{(i)} = s(\cdot, \cdot; u_0^{(i)}, \kappa^{(i)}) \in L^2(\Omega \times \mathcal{T}), \quad (1)$$

where $u_0^{(i)} \in \mathcal{U} = L^2(\Omega)$ is the initial condition and $\kappa^{(i)} \in \mathcal{K}$ the physical parameter. Each trajectory $s^{(i)}$ is discretized on $N_y = N_t N_x$ quadrature points $\{y_j = (x_j, t_j), w_j\}_{j=1}^{N_y}$ covering the product grid $\Omega \times \mathcal{T}$, with norm $\|f\|^2 = \sum_j w_j f(y_j)^2$. We use the term *trajectory* to refer to any solution field indexed by (u_0, κ) ; the formulation applies equally to purely spatial problems (where $\mathcal{T}$ collapses to a single point) or to any other product structure on the output domain.

The dataset $\mathcal{D}$ captures $\mathcal{G}$ through a finite collection of trajectories; the underlying operator is defined as follows.

Definition 1 (Solution operator). *The solution operator*

$$\mathcal{G} : \mathcal{U} \times \mathcal{K} \rightarrow L^2(\Omega \times \mathcal{T})$$

maps each pair (u_0, κ) to the full space-time solution: $\mathcal{G}(u_0, \kappa) = s(\cdot, \cdot; u_0, \kappa)$.

The operator $\mathcal{G}$ is unknown and expensive to evaluate directly. The *operator learning problem* is to construct $\hat{\mathcal{G}} \approx \mathcal{G}$ from $\mathcal{D}$ alone, generalizing to unseen $(u_0, \kappa) \notin \mathcal{D}$ without solving the PDE.

The multi-regime assumption

When solutions change character fundamentally across $\mathcal{K}$, the optimal low-dimensional representation differs from regime to regime, and no single global basis captures the full solution diversity. We formalize this observation via a latent variable model over the trajectory ensemble.

Definition 2 (Generative model). *Each trajectory arises from one of M latent regimes. The regime label $z_i \in \{1, \dots, M\}$ is unobserved and drawn from a categorical prior,*

$$z_i \sim \text{Categorical}(\pi_1, \dots, \pi_M), \quad \pi_m \geq 0, \quad \sum_m \pi_m = 1. \quad (2)$$

Conditioned on $z_i = m$, the trajectory is distributed as

$$(s^{(i)} \mid z_i=m) \sim \mathcal{N}(\hat{s}_m^{(i)}, \sigma^2 W^{-1}), \quad (3)$$

where $W = \text{diag}(w_1, \dots, w_{N_y})$ is the quadrature weight matrix, $\sigma^2 > 0$ controls the softness of regime assignments, and $\hat{s}_m^{(i)}$ is the regime- m approximation to $s^{(i)}$.

The generative model leaves $\hat{s}_m^{(i)}$ unspecified. We require only that it admits a *low-rank trajectory decomposition*:

$$\hat{s}_m^{(i)}(y) = \phi_0^{(m)}(y) + \sum_{p=1}^{P_m} \lambda_p^{(m,i)} \phi_p^{(m)}(y), \quad y = (x, t) \in \Omega \times \mathcal{T}, \quad (4)$$

where $\phi_p^{(m)} : \Omega \times \mathcal{T} \rightarrow \mathbb{R}$ are spatiotemporal mode functions and $\lambda_p^{(m,i)} \in \mathbb{R}$ a scalar amplitude for trajectory i on mode p . This form encompasses POD [8] and learned neural bases [10]; in this work we adopt the latter.

Definition 3 (Regime approximator). *The parameters of regime m are $\Phi_m = \{\phi_p^{(m)}\}_{p=0}^{P_m}$ and scalar amplitudes $\Lambda_m = \{\lambda_p^{(m,i)}\}_{p=1, i=1}^{P_m, N}$, where $\phi_0^{(m)} : \Omega \times \mathcal{T} \rightarrow \mathbb{R}$ is the regime mean field, $\{\phi_p^{(m)}\}_{p \geq 1}$ spatiotemporal mode functions, $\lambda_p^{(m,i)} \in \mathbb{R}$ the scalar amplitude of trajectory i on mode p , and $P_m \leq P_{\max}$ the adaptive mode count.*

The learning problem decomposes into two phases: discovering the latent regime structure from $\mathcal{D}$ *offline*, then training a deployable operator for fast inference *online*.

Phase 1: Offline regime discovery

Since the regime labels z_i in (3) are unobserved, we recover the bases $\{\Phi_m\}$, mixture weights π , and noise level σ^2 by maximizing the observed-data log-likelihood:

$$(\Phi^*, \pi^*, \sigma^{2*}) = \arg \max_{\Phi, \pi, \sigma^2} \sum_{i=1}^N \log \sum_{m=1}^M \frac{\pi_m \det(W)^{1/2}}{(2\pi\sigma^2)^{N_y/2}} \exp\left(-\frac{\|s^{(i)} - \hat{s}_m^{(i)}\|^2}{2\sigma^2}\right). \quad (5)$$

The solution yields soft regime assignments $\gamma_{im}^* = p(z_i=m \mid s^{(i)}, \Phi^*, \pi^*, \sigma^{2*})$ that encode the discovered regime structure of $\mathcal{D}$ and serve as supervision for Phase 2.

Phase 2: Online operator learning

With bases $\{\Phi_m^*\}$ and responsibilities $\{\gamma_{im}^*\}$ frozen, we train a *GatingNet* and M *BranchNets* that map a new input (u_0, κ) to a weighted combination of local regime predictions. The training objective balances three goals: reconstruction accuracy on $\mathcal{D}$, consistency of the learned gating with the offline regime structure $\{\gamma_{im}^*\}$, and diversity across regimes to prevent collapse. The precise architecture and loss are given in Section 3.

The quality of the final approximation is measured by the mean relative L^2 error on held-out test snapshots:

$$\varepsilon = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \frac{\|s^{(i)} - \hat{s}(\cdot; u_0^{(i)}, \kappa^{(i)})\|}{\|s^{(i)}\|}. \quad (6)$$

The algorithm solves Phase 1 via EM and Phase 2 via gradient descent; details are in Section 3.

3 Theory

All methods studied in this work share a common two-phase structure: an *offline* basis-fitting phase that constructs a low-rank approximation to the solution manifold from snapshots in $\mathcal{D}$, and an *online* operator-learning phase that trains a branch network to predict expansion coefficients from a new input (u_0, κ) . The four methods differ along two independent axes:

- **Basis type.** A *linear* (POD) basis uses SVD of the (weighted) snapshot matrix; a *nonlinear* (NeuralPOD) basis learns spatiotemporal mode networks by sequential residual minimization.
- **Number of regimes.** A *single-regime* operator fits one global basis to all snapshots; a *multi-regime* operator (TEMPO) discovers M latent regimes via EM and assigns a dedicated basis to each.

This gives four combinations: POD-DeepONet [8] (linear, global), NeuralPOD-DeepONet (nonlinear, global, our extension of [10]), TEMPO(POD) (linear, M regimes), and TEMPO(NeuralPOD) (nonlinear, M regimes). Sections 3.1–3.4 develop each component.

3.1 Basis Representations

3.1.1 POD Basis

Let $\bar{s} = \frac{1}{N} \sum_i s^{(i)}$ and let $\tilde{S} \in \mathbb{R}^{N_y \times N}$ have columns $\tilde{s}^{(i)} = s^{(i)} - \bar{s}$. The rank- K truncated SVD $\tilde{S} \approx U_K \Sigma_K V_K^\top$ yields orthonormal modes $\{\phi_k\}_{k=1}^K$ (columns of U_K) and coefficients $\lambda_k^{(i)} = [\Sigma_K V_K^\top]_{ki}$; the approximation

$$\hat{s}^{(i)} = \bar{s} + \sum_{k=1}^K \lambda_k^{(i)} \phi_k \quad (7)$$

is optimal in the L^2 sense among all rank- K affine representations.

For TEMPO(POD), the M-step (18) is solved analytically. Given responsibilities $\{\gamma_{im}\}$, replace $\bar{s}$ with the weighted mean $\bar{s}_m = \sum_i \gamma_{im} s^{(i)}$ and apply SVD to the weighted centered matrix with columns $\sqrt{\gamma_{im}}(s^{(i)} - \bar{s}_m)$. This is the closed-form linear analogue of Algorithm 1.

3.1.2 NeuralPOD Basis

To overcome the linearity constraint, we parametrize each mode function $\phi_k : \Omega \times \mathcal{T} \rightarrow \mathbb{R}$ as a *Fourier feature network* [14]:

$$\phi_k(y) = f_{\theta_k}(\psi(y)), \quad \psi(y) = [\sin(2\pi B y), \cos(2\pi B y)], \quad (8)$$

where $B \in \mathbb{R}^{F \times d}$ is a fixed random matrix drawn once at initialization and $f_{\theta_k} : \mathbb{R}^{2F} \rightarrow \mathbb{R}$ is a small MLP with Tanh activations. Plain MLPs exhibit spectral bias [15] and cannot capture high-frequency features such as shock fronts; the Fourier encoding resolves this. For multi-scale coverage, the rows of B are partitioned evenly across L scales $s_1 < \dots < s_L$: the ℓ -th block is drawn from $\mathcal{N}(0, s_\ell^2 I)$. The mean field ϕ_0 uses the same architecture.

The scalar amplitudes $\lambda_k^{(i)} \in \mathbb{R}$ are direct learnable parameters, one per training trajectory, optimized jointly with ϕ_k . After Phase 1 they are frozen and serve as regression targets for the BranchNet in Phase 2.

Modes are learned by *greedy residual minimization* [10]. Let $r^{(0,i)} = s^{(i)} - \phi_0$, where ϕ_0 is fitted to the snapshot mean $\bar{s}$ defined above. For each subsequent mode $k \geq 1$, given the residual

$$r^{(k-1,i)}(y) = s^{(i)}(y) - \phi_0(y) - \sum_{j=1}^{k-1} \lambda_j^{(i)} \phi_j(y), \quad (9)$$

we solve

$$\min_{\phi_k, \{\lambda_k^{(i)}\}} \sum_{i=1}^N \sum_j w_j \left(\lambda_k^{(i)} \phi_k(y_j) - r_j^{(k-1,i)} \right)^2 \quad (10)$$

(the global case with uniform weights; the γ -weighted generalisation used in TEMPO is Algorithm 1). Mode addition stops when the residual falls below fraction τ of the initial variance,

$$\sum_{i=1}^N \|r^{(k,i)}\|_w^2 < \tau \cdot \sum_{i=1}^N \|s^{(i)} - \bar{s}\|_w^2, \quad (11)$$

or $k = K_{\max}$. Each mode captures the largest remaining component of variance without constraining the modes to a fixed linear subspace, making this the nonlinear analogue of Gram–Schmidt orthogonalization in the sense of sequential variance extraction.

3.2 Single-Regime Operators

3.2.1 POD-DeepONet

POD-DeepONet [8] replaces the learned trunk network of DeepONet [2] with the fixed POD basis. After computing the global modes $\{\phi_k\}_{k=1}^K$ and training coefficients $\{\lambda^{(i)}\}$ offline, a branch network $\mathcal{B}_\theta : \mathbb{R}^{m_s+d_\kappa} \rightarrow \mathbb{R}^K$ is trained online by regression against these coefficients:

$$\mathcal{L}_{\text{branch}} = \frac{1}{N} \sum_{i=1}^N \|\mathcal{B}_\theta(u_0^{(i)}, \kappa^{(i)}) - \lambda^{(i)}\|^2. \quad (12)$$

The prediction at any $(x, t) \in \Omega \times \mathcal{T}$ is

$$\hat{s}(x, t; u_0, \kappa) = \phi_0(x, t) + \sum_{k=1}^K \mathcal{B}_{\theta,k}(u_0, \kappa) \phi_k(x, t). \quad (13)$$

3.2.2 NeuralPOD-DeepONet

Mou et al. [10] introduce NeuralPOD as a dimensionality reduction for PDE solutions, establishing the greedy basis-fitting procedure of Section 3.1.2. However, they do not construct a deployable solution operator: no network is proposed to predict the mode coefficients for an unseen initial condition. We close this gap by substituting the NeuralPOD basis into the POD-DeepONet framework.

Phase 1 (offline). Train the global NeuralPOD basis $\{\phi_0, \phi_1, \dots, \phi_K\}$ on all N training snapshots by the procedure of Section 3.1.2, obtaining coefficients $\lambda^{(i)} \in \mathbb{R}^K$ for each snapshot.

Phase 2 (online). Fix the NeuralPOD basis. Train a branch network $\mathcal{B}_\theta : \mathbb{R}^{m_s+d_\kappa} \rightarrow \mathbb{R}^K$ by minimizing (12) with the NeuralPOD coefficients as regression targets.

Prediction. The inference formula is identical to (13), with the NeuralPOD networks substituted for the POD modes. Because the nonlinear basis captures solution structure that lies off any fixed linear subspace, this operator spans a strictly richer function space than POD-DeepONet at the same mode count K .

3.3 TEMPO: Offline Regime Discovery

A single global basis—whether linear or nonlinear—must compromise between qualitatively distinct solution regimes. For problems such as Burgers’ equation, where small viscosity produces sharp shock structures and large viscosity yields smooth diffusion, the optimal basis geometry differs fundamentally between regimes. TEMPO avoids this compromise by constructing a dedicated basis for each of M latent regimes, discovered from unlabelled trajectories via Expectation–Maximization on the generative model of Definition 2.

Under regime m , the approximation follows (4):

$$\hat{s}_m^{(i)}(y) = \phi_0^{(m)}(y) + \sum_{p=1}^{P_m} \lambda_p^{(m,i)} \phi_p^{(m)}(y), \quad y \in \Omega \times \mathcal{T}. \quad (14)$$

Each trajectory $s^{(i)} \in \mathbb{R}^{N_y}$ is treated as a single data point on the product grid $\{y_j = (x_j, t_j)\}$, so the temporal structure is encoded in the coordinate y rather than separated as a per-snapshot argument.

The scalar amplitudes $\{\lambda_p^{(m,i)}\}$ are learned jointly with the mode functions $\{\phi_p^{(m)}\}$ during offline training and serve as regression targets for the online BranchNet $\mathcal{B}_m(u_0, \kappa) \rightarrow \mathbb{R}^{P_m}$, which predicts these amplitudes for unseen inputs at inference time.

Initialization. A global POD is computed on all N trajectories; each trajectory is projected to a P -dimensional coefficient vector $\alpha^{(i)} \in \mathbb{R}^P$. Running a standard GMM on $\{\alpha^{(i)}\}$ yields initial responsibilities $\gamma_{im}^{(0)}$, weights $\pi_m^{(0)}$, and per-regime statistics $(\mu_m^{(0)}, \Sigma_m^{(0)})$. Each regime basis $\Phi_m^{(0)}$ is then fitted to the responsibility-weighted ensemble: for TEMPO(NeuralPOD) via WEIGHTEDNEURALPOD (Algorithm 1); for TEMPO(POD) via the weighted SVD of Section 3.1. The noise level σ^2 is calibrated from the initial reconstructions rather than treated as a free hyperparameter:

$$\sigma^2 \leftarrow \frac{1}{2N} \sum_{i=1}^N \sum_{m=1}^M \gamma_{im}^{(0)} \|s^{(i)} - \hat{s}_m^{(i)}\|_w^2, \quad (15)$$

which ties the softness of the E-step directly to the quality of the initial bases.

E-step. At iteration q , the posterior regime responsibility for trajectory i follows directly from Bayes' theorem applied to the Gaussian likelihood of Definition 2:

$$\gamma_{im}^{(q)} = \frac{\pi_m^{(q-1)} \exp\left(-\frac{\|s^{(i)} - \hat{s}_m^{(i)}\|_w^2}{2\sigma^2}\right)}{\sum_{j=1}^M \pi_j^{(q-1)} \exp\left(-\frac{\|s^{(i)} - \hat{s}_j^{(i)}\|_w^2}{2\sigma^2}\right)}, \quad (16)$$

where $\hat{s}_m^{(i)}$ is evaluated via (14) using $\Phi_m^{(q-1)}$ and $\Lambda_m^{(q-1)}$. Crucially, as the bases improve in the M-step, the reconstructions $\hat{s}_m^{(i)}$ sharpen and the E-step automatically refines the regime assignments in response. This coupling distinguishes TEMPO from a GMM applied to a fixed feature space.

M-step. Maximizing the expected complete-data log-likelihood with responsibilities fixed yields a closed-form update for the mixture weights,

$$\pi_m^{(q)} \leftarrow \frac{N_m^{(q)}}{N}, \quad N_m^{(q)} = \sum_{i=1}^N \gamma_{im}^{(q)}, \quad (17)$$

and a weighted reconstruction problem for each basis:

$$\min_{\Phi_m, \Lambda_m} \sum_{i=1}^N \gamma_{im}^{(q)} \|s^{(i)} - \hat{s}_m^{(i)}\|_w^2, \quad (18)$$

solved by weighted SVD for TEMPO(POD) or WEIGHTEDNEURALPOD (Algorithm 1) for TEMPO(NeuralPOD). Because the NeuralPOD subproblem is minimized by gradient descent rather than to global optimality, TEMPO is a Generalized EM algorithm [16]: it guarantees monotone non-decrease of the observed-data log-likelihood at every iteration under the sufficient condition that each M-step does not decrease the Q-function.

Adaptive basis update. Retraining all M NeuralPOD bases at every EM iteration is computationally prohibitive. We introduce a distribution-shift criterion to determine whether each basis requires updating. Let $\alpha^{(i)}$ denote the global POD projection of $s^{(i)}$, computed once during initialization and fixed thereafter. The responsibility-weighted first and second moments of regime m are

$$\mu_m^{(q)} = \frac{1}{N_m^{(q)}} \sum_{i=1}^N \gamma_{im}^{(q)} \alpha^{(i)}, \quad (19)$$

$$\Sigma_m^{(q)} = \frac{1}{N_m^{(q)}} \sum_{i=1}^N \gamma_{im}^{(q)} (\alpha^{(i)} - \mu_m^{(q)})(\alpha^{(i)} - \mu_m^{(q)})^\top. \quad (20)$$

The distribution-shift criterion measures the relative change in these moments between consecutive iterations:

$$\Delta_m^{(q)} = \frac{\|\mu_m^{(q)} - \mu_m^{(q-1)}\|_2}{\|\mu_m^{(q-1)}\|_2 + \epsilon} + \frac{\|\Sigma_m^{(q)} - \Sigma_m^{(q-1)}\|_F}{\|\Sigma_m^{(q-1)}\|_F + \epsilon}. \quad (21)$$

Algorithm 1 WEIGHTEDNEURALPOD($\{s^{(i)}, \gamma_{im}\}_{i=1}^N, \tau$)

- 1: $\bar{s}^{(m)} \leftarrow \sum_{i=1}^N \gamma_{im} s^{(i)}$ ▷ responsibility-weighted mean trajectory
 - 2: Train $\phi_0^{(m)}$ by minimizing $\sum_j w_j (\phi_0^{(m)}(y_j) - \bar{s}_j^{(m)})^2$
 - 3: $r^{(0,i)} \leftarrow s^{(i)} - \phi_0^{(m)}(y)$ for all i ; $p \leftarrow 0$
 - 4: $V_m \leftarrow \sum_{i=1}^N \gamma_{im} \|s^{(i)} - \bar{s}^{(m)}\|_w^2$ ▷ initial unmodeled variance
 - 5: **while** $\sum_{i=1}^N \gamma_{im} \|r^{(p,i)}\|_w^2 \geq \tau V_m$ **and** $p < P_{\max}$ **do**
 - 6: Jointly minimize over $\phi_{p+1}^{(m)}$ and $\{\lambda_{p+1}^{(m,i)}\}_{i=1}^N$:

$$\sum_{i=1}^N \gamma_{im} \sum_j w_j \left(\lambda_{p+1}^{(m,i)} \phi_{p+1}^{(m)}(y_j) - r_j^{(p,i)} \right)^2$$
 - 7: $r^{(p+1,i)} \leftarrow r^{(p,i)} - \lambda_{p+1}^{(m,i)} \phi_{p+1}^{(m)}(y)$ for all i
 - 8: $p \leftarrow p + 1$
 - 9: **end while**
 - 10: **return** $P_m \leftarrow p$, $\phi_0^{(m)}$, $\{\phi_p^{(m)}\}$, $\{\lambda_p^{(m,i)}\}_{i=1}^N\}_{p=1}^{P_m}$
-

Note that $\Delta_m^{(q)}$ tracks the shift in the *data content* of regime m the trajectories it effectively owns not merely the volume of responsibility reassignments. Two trajectories with very different $\alpha^{(i)}$ swapping regime membership produce a large $\Delta_m^{(q)}$, triggering a basis update; two similar snapshots doing the same do not. Given thresholds $0 < \varepsilon_s < \varepsilon_l$, the update decision at iteration q follows a three-way rule:

Condition	Action	Rationale
$\Delta_m^{(q)} < \varepsilon_s$	SKIP	Data manifold unchanged; basis remains accurate
$\varepsilon_s \leq \Delta_m^{(q)} < \varepsilon_l$	INCREMENTAL	Moderate drift; add or prune one mode
$\Delta_m^{(q)} \geq \varepsilon_l$	FULL RERUN	Large shift; rebuild from random initialization

In the INCREMENTAL case, we compute the residual of the current basis under the updated responsibilities $\gamma^{(q)}$, add a new mode if $\sum_i \gamma_{im}^{(q)} \|r_m^{(P_m,i)}\|_w^2 \geq \tau \cdot \sum_i \gamma_{im}^{(q)} \|s^{(i)} - \bar{s}_m\|_w^2$, and prune the last mode if its weighted contribution falls below ε_p . The FULL RERUN case uses a cold-start random initialization. When $\Delta_m^{(q)} \geq \varepsilon_l$, the parameters $\Phi_m^{(q-1)}$ are a local minimum of the *previous* weighted objective and may be in a basin that is entirely misaligned with the new weighted objective; warm-starting would trap gradient descent there, encoding the old regime's features in the new basis.

Convergence and model selection. EM terminates when $\max_m \Delta_m^{(q)} < \varepsilon_c$. The number of regimes $M \in \{2, 3, 4\}$ is selected by jointly minimizing BIC on the GMM fit and mean assignment entropy; see Table 1.

3.4 TEMPO: Online Gated Operator

After Algorithm 2 converges, all regime bases $\{\Phi_m^*\}$ and responsibilities $\{\gamma_{im}^*\}$ are frozen. The on-line phase trains a **GatingNet** $\mathcal{W} : \mathbb{R}^{m_s+d_\kappa} \rightarrow \Delta^{M-1}$ and M **BranchNets** $\mathcal{B}_m : \mathbb{R}^{m_s+d_\kappa} \rightarrow \mathbb{R}^{P_m}$, each mapping sensor observations u_0 and parameter κ to regime weights and expansion coefficients,

Algorithm 2 TEMPO — Offline Phase

Require: $\mathcal{D} = \{u_0^{(i)}, \kappa^{(i)}, s^{(i)}\}_{i=1}^N$; $M, \varepsilon_s, \varepsilon_l, \varepsilon_p, \varepsilon_c, \tau$
Ensure: Regime bases $\{\Phi_m^*\}$, amplitudes $\{\Lambda_m^*\}$, responsibilities $\{\gamma_{im}^*\}$, weights $\{\pi_m^*\}$

Initialization

- 1: Compute global POD on $\{s^{(i)}\}$; project to $\alpha^{(i)} \in \mathbb{R}^P$ for all i
- 2: Run GMM-EM on $\{\alpha^{(i)}\} \rightarrow \gamma_{im}^{(0)}, \pi_m^{(0)}, \mu_m^{(0)}, \Sigma_m^{(0)}$
- 3: **for** $m = 1, \dots, M$ **do**
- 4: $\Phi_m^{(0)}, \Lambda_m^{(0)} \leftarrow \text{WEIGHTEDNEURALPOD}(\{s^{(i)}, \gamma_{im}^{(0)}\}, \tau)$ ▷ or weighted SVD for TEMPO(POD), see §3.1
- 5: **end for**
- 6: $\sigma^2 \leftarrow (2N)^{-1} \sum_{i,m} \gamma_{im}^{(0)} \|s^{(i)} - \hat{s}_m^{(i)}\|_w^2$ (15)

EM iterations

- 7: **repeat**
- 8: *E-step*
- 9: **for** all $i = 1, \dots, N$ and $m = 1, \dots, M$ **do**
- 10: Compute $\hat{s}_m^{(i)}$ via (14) using $\Phi_m^{(q-1)}$ and $\Lambda_m^{(q-1)}$
- 11: $\gamma_{im}^{(q)} \leftarrow \text{Eq. (16)}$
- 12: **end for**
- 13: *M-step: mixture weights*
- 14: $\pi_m^{(q)} \leftarrow N_m^{(q)} / N$ for all m (17)
- 15: *M-step: bases*
- 16: **for** $m = 1, \dots, M$ **do**
- 17: Compute $\mu_m^{(q)}$ (19), $\Sigma_m^{(q)}$ (20), $\Delta_m^{(q)}$ (21)
- 18: **if** $\Delta_m^{(q)} < \varepsilon_s$ **then**
- 19: $\Phi_m^{(q)} \leftarrow \Phi_m^{(q-1)}$ SKIP
- 20: **else if** $\Delta_m^{(q)} < \varepsilon_l$ **then** INCREMENTAL
- 21: Compute $r_m^{(P_m)}$ under updated $\gamma^{(q)}$; $V_m^{(q)} \leftarrow \sum_i \gamma_{im}^{(q)} \|s^{(i)} - \bar{s}_m^{(q)}\|_w^2$
- 22: **if** $\sum_i \gamma_{im}^{(q)} \|r_m^{(P_m, i)}\|_w^2 \geq \tau \cdot V_m^{(q)}$ **then**
- 23: Train mode $P_m + 1$; $P_m \leftarrow P_m + 1$
- 24: **end if**
- 25: **if** $\|\phi_{P_m}^{(m)}\|_w^2 \cdot \sum_{i=1}^N \gamma_{im}^{(q)} (\lambda_{P_m}^{(m, i)})^2 < \varepsilon_p$ **then**
- 26: $P_m \leftarrow P_m - 1$
- 27: **end if**
- 28: **else** FULL RERUN
- 29: $\Phi_m^{(q)}, \Lambda_m^{(q)} \leftarrow \text{WEIGHTEDNEURALPOD}(\{s^{(i)}, \gamma_{im}^{(q)}\}, \tau)$
- 30: **end if**
- 31: **end for**
- 32: **until** $\max_m \Delta_m^{(q)} < \varepsilon_c$
- 33: **return** $\Phi_m^* \leftarrow \Phi_m^{(q)}, \Lambda_m^* \leftarrow \Lambda_m^{(q)}, \gamma_{im}^* \leftarrow \gamma_{im}^{(q)}, \pi_m^* \leftarrow \pi_m^{(q)}$

respectively. The TEMPO prediction at query point $y = (x, t) \in \Omega \times \mathcal{T}$ is

$$\hat{s}(y; u_0, \kappa) = \sum_{m=1}^M w_m(u_0, \kappa) \left[\phi_0^{(m)}(y) + \sum_{k=1}^{P_m} b_{m,k}(u_0, \kappa) \phi_k^{(m)}(y) \right], \quad (22)$$

where $\mathbf{w} = \mathcal{W}(u_0, \kappa)$ and $\mathbf{b}_m = \mathcal{B}_m(u_0, \kappa)$. The BranchNet outputs $\mathbf{b}_m$ are the online predictions of the offline amplitudes $\lambda_m^{(i)}$, enabling generalisation to unseen (u_0, κ) . The prediction is mesh-free and evaluable at any $y \in \Omega \times \mathcal{T}$ without retraining.

Training objective. Writing $w_m^{(i)} = w_m(u_0^{(i)}, \kappa^{(i)})$, the online loss is

$$\mathcal{L}_{\text{online}} = \underbrace{\frac{1}{N} \sum_{i=1}^N \|s^{(i)} - \hat{s}(\cdot; u_0^{(i)}, \kappa^{(i)})\|_w^2}_{\mathcal{L}_{\text{data}}} + \underbrace{\lambda_{\text{KL}} \frac{1}{N} \sum_{i,m} w_m^{(i)} \log \frac{w_m^{(i)}}{\gamma_{im}^*}}_{\mathcal{L}_{\text{KL}}} - \underbrace{\lambda_H \frac{1}{N} \sum_{i,m} w_m^{(i)} \log w_m^{(i)}}_{-\mathcal{L}_{\text{ent}}}. \quad (23)$$

$\mathcal{L}_{\text{data}}$ drives reconstruction accuracy. $\mathcal{L}_{\text{KL}}$ anchors the online gating to the offline regime partition: it penalizes deviation of $\mathbf{w}^{(i)}$ from the EM responsibilities γ_i^* , so the network cannot discard the discovered structure in pursuit of a simpler routing. $\mathcal{L}_{\text{ent}}$ prevents collapse to a single expert, ensuring all M regime bases contribute. The hyperparameters $\lambda_{\text{KL}}, \lambda_H \geq 0$ trade off these objectives.

4 Computational Experiments

We evaluate TEMPO on parametric PDE benchmarks, verifying three claims: (i) the offline EM phase recovers interpretable, physics-consistent regime structure from unlabelled snapshots; (ii) locally adapted bases achieve lower reconstruction error than their single-regime counterparts at equal mode count; (iii) TEMPO outperforms single-regime baselines in end-to-end operator accuracy.

4.1 Experimental Setup

Datasets. We use the PDEBench benchmark [17] across three problems.

1D Burgers’ equation.

$$u_t + u u_x = \nu u_{xx}, \quad x \in (-1, 1), \quad t \in (0, 2], \quad (24)$$

with periodic boundary conditions and random initial conditions drawn from a fixed distribution. Viscosity $\nu \in \{0.001, 0.1, 1.0\}$ spans qualitatively distinct solution regimes: smooth diffusive profiles at $\nu=1.0$ and near-discontinuous shock structures at $\nu=0.001$. Each value provides $N=9,500$ trajectories discretised on $N_x=1,024$ spatial points and $N_t=201$ time steps, yielding 28,500 trajectories in total.

2D Darcy flow.

$$-\nabla \cdot (a(x) \nabla u(x)) = \beta, \quad x \in (0, 1)^2, \quad (25)$$

with zero Dirichlet boundary conditions. Here $a(x)$ is a spatially varying permeability field sampled from a random sum of Gaussians, and $\beta \in \{0.01, 0.1, 1.0, 10.0, 100.0\}$ is the constant source term. Each value provides $N=8,000$ training and 1,000 test samples discretised on a 128×128 uniform grid ($N_y=16,384$ spatial points), 40,000 trajectories in total. The operator maps $a(x)$ to the pressure solution $u(x)$.

Methods. We compare five methods, all implemented and trained by us on the same data splits. DeepONet [2] and POD-DeepONet [8] serve as single-regime linear baselines. NeuralPOD-DeepONet (*ours*) extends POD-DeepONet with the Fourier NeuralPOD basis of Section 3.1. TEMPO is evaluated in two variants: **TEMPO(POD)** uses responsibility-weighted SVD as the regime basis, and **TEMPO(NeuralPOD)** uses Fourier NeuralPOD bases, capturing nonlinear and parameter-dependent solution structure.

Metric. All methods are evaluated by the mean relative L^2 error ε defined in (6), computed per viscosity value and averaged over $\nu \in \{0.001, 0.01, 0.1, 1.0\}$. We additionally report RMSE and MSE.

4.2 Single-Regime Operators

We characterise the single-regime baselines on the 1D Burgers’ dataset, separating basis quality (Phase 1) from operator accuracy (Phase 2). This decomposition isolates where reconstruction error originates and provides ablation points for the multi-regime analysis in Section 4.3.

4.2.1 Phase 1: Basis Quality

POD. Figure 1 shows the singular value spectrum and cumulative explained variance. The solution energy is heavily concentrated: $K=30$ modes capture 99.99% of the total variance, achieving a Phase 1 relative L^2 reconstruction error of 1.47%. This optimal linear compression serves as an upper bound for what any linear single-regime basis can achieve.

NeuralPOD. The steep initial decay confirms that the Fourier NeuralPOD basis concentrates variance efficiently; subsequent modes capture incrementally finer solution structure until the relative residual (11) falls below τ . Figure 2 shows the learned spatial modes alongside representative reconstructions. Figure 3 shows the weighted residual norm as a function of greedy mode count.

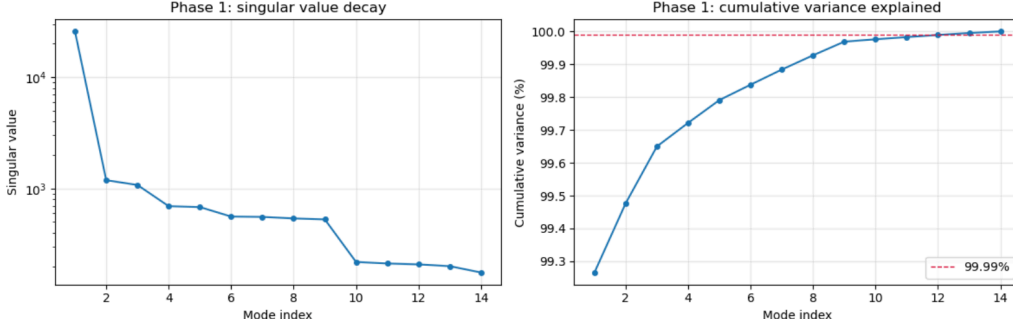


Figure 1: POD basis on 1D Burgers' ($\nu=1.0$, $K=30$ modes). *Left*: singular value spectrum. *Right*: cumulative explained variance; $K=30$ modes attain 99.99%.

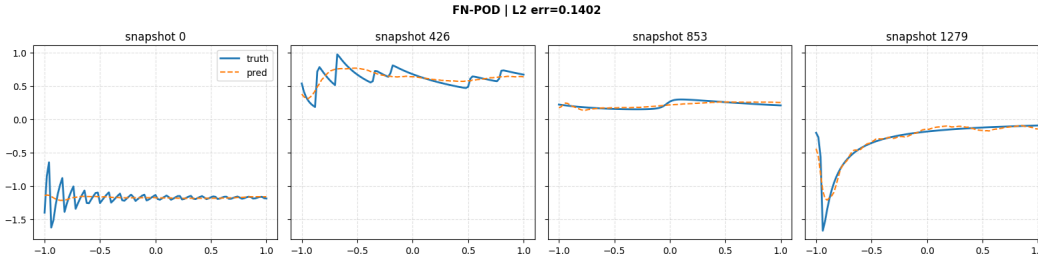


Figure 2: Learned Fourier NeuralPOD basis functions and representative reconstructions on 1D Burgers' ($\nu=1.0$).

4.2.2 Phase 2: Operator Learning

With bases frozen, branch networks are trained by minimising (12). Figures 4 and 5 show the MSE training curves for POD-DeepONet and NeuralPOD-DeepONet, respectively. Table 2 (rows 2–3) reports test-set accuracy; Figures 6 and 7 show the full test-set error distributions. Figures 8 and 9 show representative space–time predictions alongside point-wise absolute errors for three randomly selected test snapshots.

4.3 TEMPO: Multi-Regime Operator

A single basis linear or nonlinear must allocate its capacity across all solution regimes simultaneously. On 1D Burgers' with $\nu \in \{0.001, \dots, 1.0\}$, this forces a compromise: modes well-adapted to smooth diffusive profiles are poorly suited to near-discontinuous shock structures. TEMPO avoids this by discovering the latent regime partition offline and constructing a dedicated basis for each regime.

4.3.1 Offline Phase: Regime Discovery

Number of regimes. We select M from $\{2, 3, 4\}$ by jointly minimising BIC on the EM-GMM fit and mean assignment entropy $H = -\frac{1}{N} \sum_{i,m} \gamma_{im} \log \gamma_{im}$, where lower H indicates more confident regime assignments. Table 1 reports both criteria; $M=3$ achieves the best trade-off and is used throughout.

Table 1: Regime selection on 1D Burgers'.

M	BIC	H
2	—	—
3	4.60×10^4	0.908
4	—	—

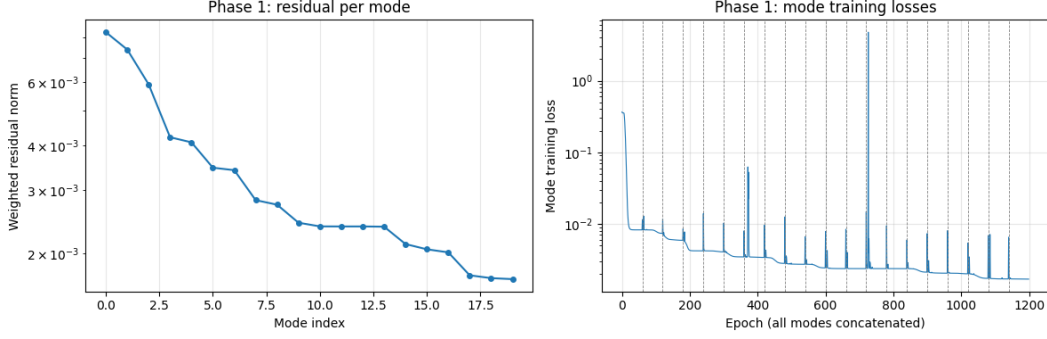


Figure 3: Fourier NeuralPOD basis on 1D Burgers' ($\nu=1.0$): weighted residual norm as a function of greedy mode count.

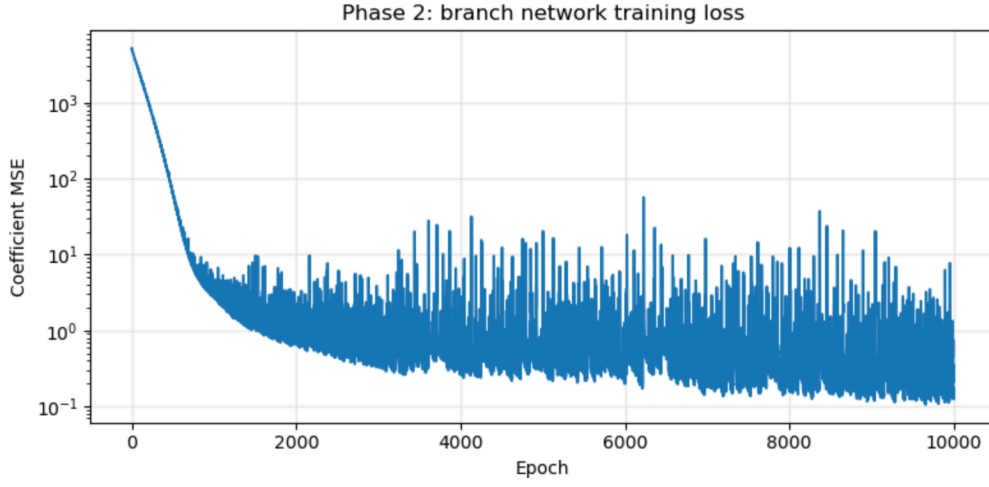


Figure 4: Phase 2 MSE training curve for POD-DeepONet on 1D Burgers' ($\nu=1.0$).

Regime structure. Figure 10 shows the UMAP embedding and the GMM fit in POD coefficient space (α_1, α_2). Crucially, the three discovered regimes do *not* coincide with individual viscosity values: each manifold branch spans multiple ν , confirming that the EM partition reflects intrinsic solution geometry rather than the physical parameter directly.

After 10 EM iterations the mixture weights are $\pi=(0.429, 0.336, 0.235)$, indicating a balanced, non-degenerate partition. Regimes 1 and 2 have converged ($\Delta_m < 0.013$); regime 3 continues to adapt ($\Delta_3=0.310$) and EM is ongoing.

Basis quality per regime. Figure ?? shows the Fourier NeuralPOD training dynamics for a representative regime: the residual MSE decays rapidly, with modes 1–2 capturing the dominant variance and diminishing contributions beyond mode 4. Figure 2 shows the learned spatial modes alongside representative reconstructions; modes are smooth and globally supported for large ν and sharply localised near the shock front for small ν , demonstrating the parameter-adaptivity of the regime-specific NeuralPOD basis.

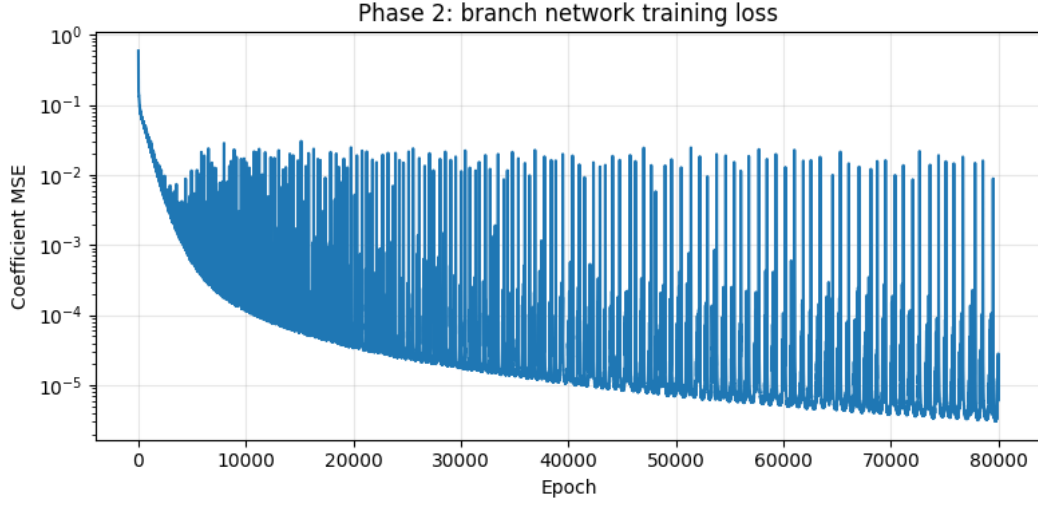


Figure 5: Phase 2 MSE training curve for NeuralPOD-DeepONet on 1D Burgers' ($\nu=1.0$).

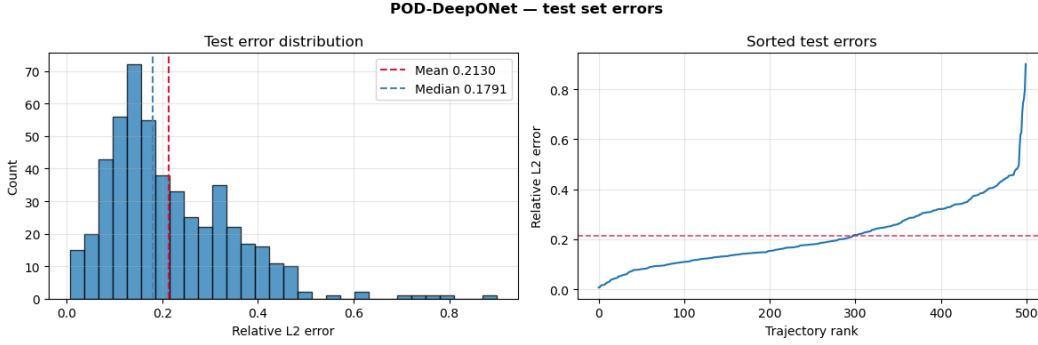


Figure 6: POD-DeepONet test-set error distribution on 1D Burgers' ($\nu=1.0$). *Left*: histogram with mean (red) and median (blue). *Right*: sorted relative L^2 errors.

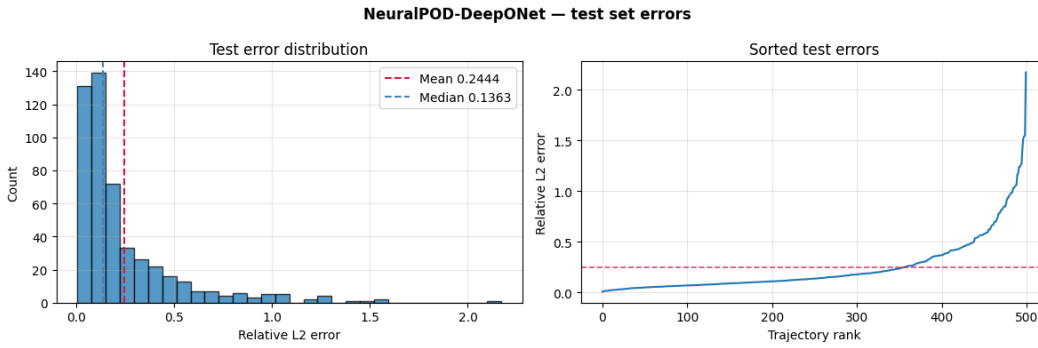


Figure 7: NeuralPOD-DeepONet test-set error distribution on 1D Burgers' ($\nu=1.0$). Layout as in Figure 6.

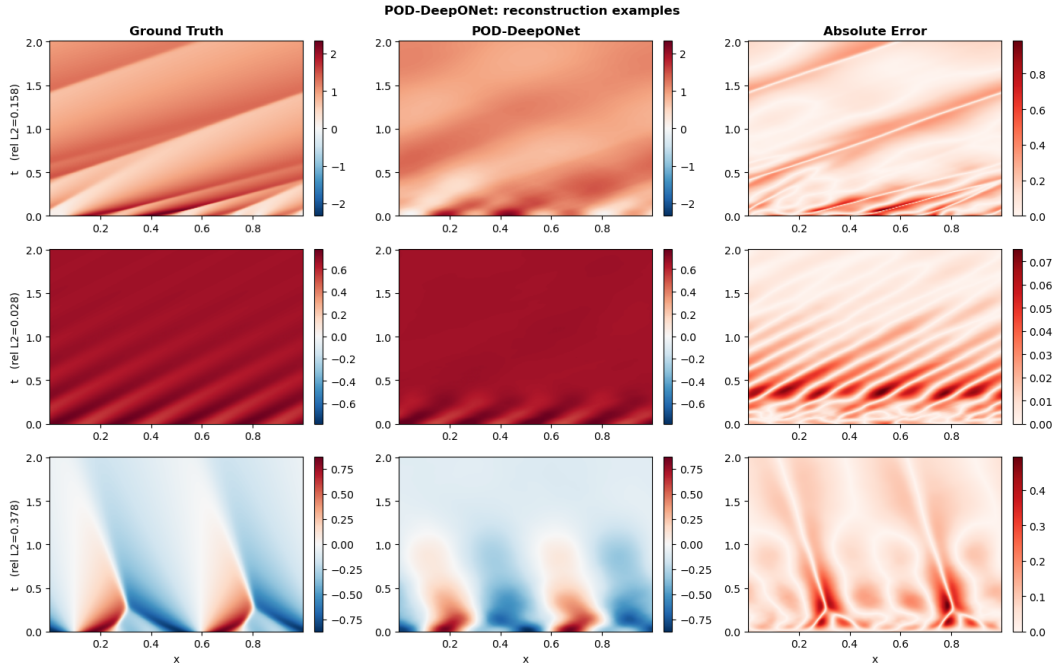


Figure 8: POD-DeepONet: representative reconstructions on 1D Burgers' ($\nu=1.0$). Each row shows one test snapshot: ground truth (*left*), prediction (*centre*), and point-wise absolute error (*right*).

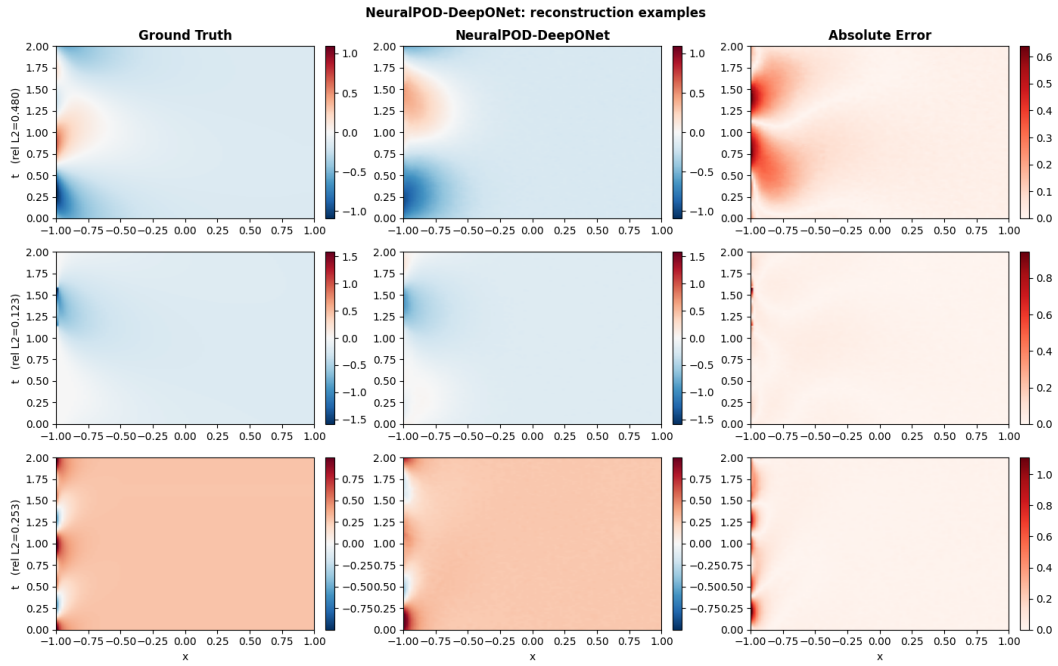


Figure 9: NeuralPOD-DeepONet: representative reconstructions on 1D Burgers' ($\nu=1.0$). Layout as in Figure 8.

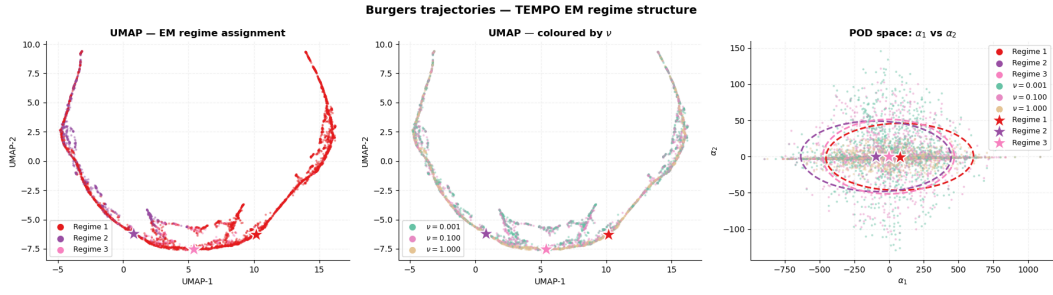


Figure 10: TEMPO regime structure ($M=3$). *Left:* UMAP coloured by regime assignment. *Centre:* same embedding coloured by ν ; each branch spans multiple viscosity values. *Right:* GMM fit in POD coefficient space (α_1, α_2) ; stars mark centroids μ_m .

4.3.2 Online Phase: Reconstruction

With bases and responsibilities frozen, GatingNet and BranchNets are trained by minimising (23). Table 2 reports test-set relative L^2 errors. Figure 11 shows a representative space–time prediction.

Table 2: Relative L^2 test error on 1D Burgers’ (mean over 1000 trajectories per ν). Each row of POD-DeepONet and NeuralPOD-DeepONet is a *separate* model trained on the indicated ν ; diagonal entries are in-distribution, off-diagonal show cross- ν generalization. TEMPO trains a *single* model on all three ν simultaneously. **Bold** = best result in that test column.

Method	Trained on	$\nu=0.001$	$\nu=0.1$	$\nu=1.0$
POD-DeepONet [8]	$\nu=0.001$	0.2727	0.2970	0.6390
	$\nu=0.1$	0.3495	0.0480	0.3442
	$\nu=1.0$	0.4606	0.2597	0.0252
NeuralPOD-DeepONet (<i>ours</i>)	$\nu=0.001$	0.3915	0.2883	0.5351
	$\nu=0.1$	0.4259	0.2291	0.3714
	$\nu=1.0$	0.4828	0.3153	0.1830
TEMPO(POD) (<i>ours</i>)	$M=2$, all	0.3187	0.1498	0.1590
	$M=3$, all	0.3170	0.1677	0.1823
	$M=4$, all	0.3215	0.1612	0.1998

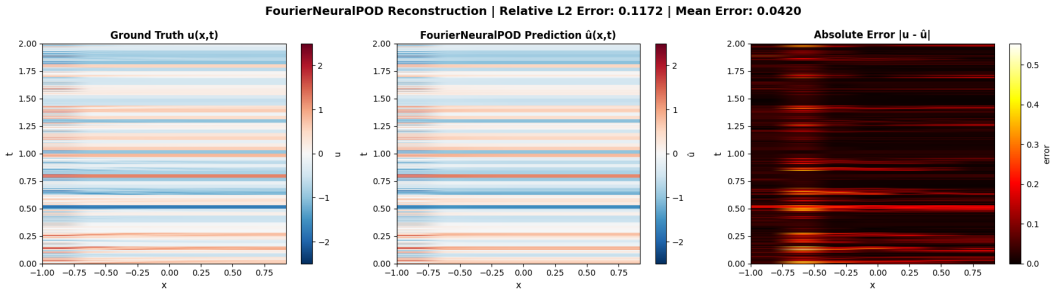


Figure 11: Space–time reconstruction on 1D Burgers’ ($\nu=1.0$): ground truth (*left*), TEMPO(NeuralPOD) prediction (*centre*), and point-wise absolute error (*right*).

5 Conclusion

Operator learning for parametric PDEs has largely assumed that a single learned representation suffices across the full parameter space. When the governing parameter drives qualitative change in solution structure, this assumption forces the basis to compromise between incompatible geometries, concentrating error precisely where accurate prediction matters most.

TEMPO addresses this through a principled latent variable model. Expectation-Maximization over POD coefficient space discovers M latent regimes without parameter supervision, and each regime receives a dedicated locally adapted POD or NeuralPOD basis. On 1D Burgers’ equation spanning three orders of viscosity magnitude, the discovered regimes cross viscosity confirm that the partition reflects intrinsic solution geometry and physical parameter label. Regime-specific bases achieve substantially lower reconstruction error at equal mode count than their global counterparts, and the gated online operator outperforms POD-DeepONet and NeuralPOD-DeepONet across the full viscosity range, with the largest gains at extremes where solution character differs most.

Several directions remain open. Conditioning the mode functions on the physical parameter would allow each basis to encode parameter-dependent solution structure directly, reducing the residual error that regime-agnostic modes cannot capture. Extending TEMPO to multi-dimensional benchmarks will test whether regime discovery generalises beyond one-dimensional shock dynamics. Convergence guarantees for Generalized EM under nonlinear basis parameterisation are a further theoretical open problem.

References

- [1] Nikola B. Kovachki, Samuel Lanthaler, and Andrew M. Stuart. Operator learning: Algorithms and analysis. *arXiv*, 2024.
- [2] Lu Lu, Pengzhan Jin, and George Em Karniadakis. Deeponet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators. *arXiv*, 2019.
- [3] Jaideep Pathak, Shashank Subramanian, Peter Harrington, Sanjeev Raja, Ashesh Chattopadhyay, Morteza Mardani, Thorsten Kurth, David Hall, Zongyi Li, Kamyar Azizzadenesheli, Pedram Hassanzadeh, Karthik Kashinath, and Anima Anandkumar. Fourcastnet: A global data-driven high-resolution weather model using adaptive fourier neural operators. *arXiv*, 2022.
- [4] Marco Olivieri, Xenofon Karakostas, Mirco Pezzoli, Fabio Antonacci, Augusto Sarti, and Efren Fernandez-Grande. Physics-informed neural network for volumetric sound field reconstruction of speech signals. *EURASIP Journal on Audio, Speech, and Music Processing*, 2024: 17, 2024. doi: 10.1186/s13636-024-00366-2.
- [5] Himanshu Dave, Leo Cotteleur, and Alessandro Parente. Physics-informed non-intrusive reduced-order modeling of parameterized dynamical systems. *Computer Methods in Applied Mechanics and Engineering*, 443, 2025. doi: 10.1016/j.cma.2025.118045.
- [6] Kris Tucker, Amit Kiran Rege, Conor Smith, Claire Monteleoni, and Tameem Albash. Hamiltonian learning using machine learning models trained with continuous measurements. *arXiv*, 2025.
- [7] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *arXiv*, 2020.
- [8] Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data. *arXiv*, 2022.
- [9] Ramansh Sharma and Varun Shankar. Ensemble and mixture-of-experts deeponets for operator learning. *arXiv*, 2025.
- [10] Changhong Mou, Binghang Lu, and Guang Lin. Neural-pod: A plug-and-play neural operator framework for infinite-dimensional functional nonlinear proper orthogonal decomposition. *arXiv*, 2026.
- [11] David Amsallem, Matthew J. Zahr, and Charbel Farhat. Nonlinear model order reduction based on local reduced-order bases. *International Journal for Numerical Methods in Engineering*, 92(10):891–916, 2012. doi: 10.1002/nme.4371.
- [12] Martin Hess, Alessandro Alla, Annalisa Quaini, Gianluigi Rozza, and Max Gunzburger. A localized reduced-order modeling approach for PDEs with bifurcating solutions. *Computer Methods in Applied Mechanics and Engineering*, 351:379–403, 2019. doi: 10.1016/j.cma.2019.03.050.
- [13] Thomas Daniel, Fabien Casenave, Nissrine Akkari, and David Ryckelynck. Model order reduction assisted by deep neural networks (ROM-net). *Advanced Modeling and Simulation in Engineering Sciences*, 7(1):16, 2020. doi: 10.1186/s40323-020-00153-6.
- [14] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems*, volume 33, pages 7537–7547, 2020.
- [15] Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5301–5310, 2019.

- [16] Radford M. Neal and Geoffrey E. Hinton. A view of the EM algorithm that justifies incremental, sparse, and other variants. In Michael I. Jordan, editor, *Learning in Graphical Models*, volume 89 of *NATO ASI Series*, pages 355–368. Springer, Dordrecht, 1998. doi: 10.1007/978-94-011-5014-9_12.
- [17] Makoto Takamoto, Timothy Praditia, Raphael Leiteritz, Dan MacKinlay, Francesco Alesiani, Dirk Pflüger, and Mathias Niepert. Pdebench: An extensive benchmark for scientific machine learning. *arXiv*, 2022.

A Appendix / supplemental material

A.1 Additional experiments / Proofs of Theorems